

ELECTROSAFE

Manual técnico de la plataforma

Arquitectura, datos y despliegue de ElectroSafe, plataforma
de gestión del electrobisturí Valleylab Force FX-C

Programa: Ingeniería

Biomédica Universidad:

Universidad ECCI

Contenido

1. Objetivo y alcance	3
1.1 Resumen del sistema	3
2. Arquitectura	3
2.1 Capas	4
2.2 Flujo de una acción	4
3. Estructura del repositorio	4
4. Tecnologías y dependencias.....	5
4.1 Bibliotecas y servicios externos	5
4.2 Funciones del navegador utilizadas.....	5
5. Modelo de datos	5
5.1 Datos iniciales (semillas).....	6
5.2 Relación con el modelo propuesto	6
5.3 Exportación a SQL.....	6
6. Persistencia y copias de seguridad	6
6.1 Lectura y escritura	6
6.2 Exportar, importar y cargar datos del repositorio	7
7. Lógica de los módulos.....	7
7.1 Formularios definidos por datos	7
7.2 Crear, editar y eliminar	7
7.3 Seguridad del contenido mostrado	7
7.4 Checklist.....	8
7.5 Evidencias fotográficas	8
7.6 Documentos y visor	8
8. Indicadores de gestión.....	8
9. Asistente IA (búsqueda sobre la documentación).....	9
10. Código QR de la hoja de vida.....	9
10.1 Generación	9
10.2 Lectura.....	10

11. Reportes e informe técnico	10
12. Aplicación instalable (PWA)	10
13. Seguridad y privacidad	11
14. Despliegue en GitHub Pages.....	11
14.1 Publicación.....	11
14.2 Verificación posterior a la publicación.....	11
15. Guía de modificación del código.....	12
16. Limitaciones y trabajo futuro	12
17. Documentos de referencia del proyecto	13

1. Objetivo y alcance

Este manual describe cómo está construida la plataforma ElectroSafe y cómo mantenerla, modificarla y publicarla. Está dirigido a quien deba entender el código: integrantes del grupo, el docente o un futuro desarrollador. Para el uso de la plataforma desde el punto de vista del usuario, consulte el Manual de usuario.

La descripción se basa en el código de `plataforma.html` e `index.html` de la versión entregada. Donde un elemento no pudo verificarse en el código, se indica explícitamente como pendiente.

1.1 Resumen del sistema

Aspecto	Descripción
Tipo de aplicación	Aplicación web de cliente (sin servidor propio), instalable como PWA
Lenguajes	HTML5, CSS3 y JavaScript sin frameworks
Almacenamiento	localStorage del navegador, clave <code>electrosafe_db</code>
Archivos principales	<code>index.html</code> (documentación) y <code>plataforma.html</code> (aplicación operativa)
Publicación	GitHub Pages (sitio estático)
Equipo modelado	Valleylab Force FX-C (Medtronic)

2. Arquitectura

ElectroSafe es una aplicación **100 % del lado del cliente**: todo el código se descarga al navegador, que ejecuta la lógica y guarda los datos. El servidor (GitHub Pages) solo entrega archivos estáticos. Esto coincide con lo planteado en el documento 07_Arquitectura, que contempla un backend como evolución futura.

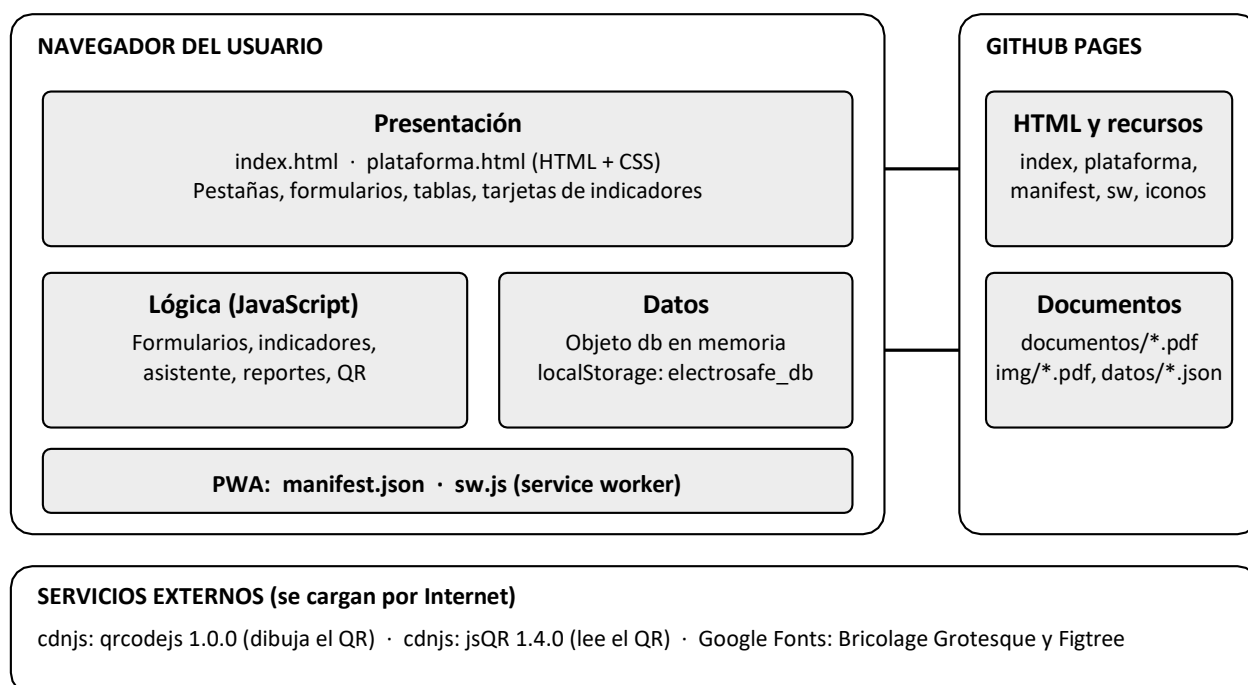


Figura 1. Componentes de ElectroSafe y su relación. Las líneas entre navegador y GitHub Pages representan la descarga de archivos.

2.1 Capas

Capa	Implementación	Responsabilidad
Presentación	HTML y CSS (variables CSS, grid, modo oscuro con <i>prefers-color-scheme</i>)	Mostrar pestañas, formularios, tablas, KPI y gráficas de barras
Lógica	JavaScript en un solo bloque dentro de <i>plataforma.html</i>	Crear formularios, guardar, calcular indicadores, buscar, generar reportes y QR
Datos	Objeto <i>db</i> + <i>localStorage</i>	Persistir los registros en el navegador
Documental	Carpetas <i>documentos/</i> e <i>img/</i> + visor en <i>iframe</i>	Servir y mostrar los PDF del proyecto
PWA	<i>manifest.json</i> y <i>sw.js</i>	Instalación como aplicación y soporte de caché

2.2 Flujo de una acción

Cada acción del usuario sigue el mismo ciclo, lo que simplifica el mantenimiento del código:

- 1 El usuario envía un formulario o pulsa un botón (evento *submit* o *click*).
- 2 El manejador modifica el objeto *db* (agrega, reemplaza o elimina un registro).
- 3 `save()` serializa *db* a JSON y lo escribe en *localStorage* dentro de un `try/catch`.
- 4 `render()` vuelve a dibujar todas las tablas, galerías, indicadores y el QR a partir de *db*.

3. Estructura del repositorio

Los archivos se publican en la raíz del repositorio, con los documentos en subcarpetas:

Electro_Safe/	
index.html	página de documentación
plataforma.html	aplicación operativa (HTML + CSS + JS)
manifest.json	metadatos de la PWA
sw.js	service worker
icon-192.png, icon-512.png	iconos de la PWA
documentos/	PDF 01 a 12 y 14
img/	electrobisturi.jpg y diagramas PDF
datos/	electrosafe-datos.json (opcional)

Importante: Las rutas son sensibles a mayúsculas en GitHub Pages. Los nombres de los PDF deben coincidir exactamente con los que usa el código (por ejemplo, `documentos/02_Normativa.pdf`). Un nombre distinto hace que el visor salga en blanco.

4. Tecnologías y dependencias

4.1 Bibliotecas y servicios externos

Recurso	Origen	Uso	Si falla
qrcodejs 1.0.0	cdnjs.cloudflare.com	Dibujar el QR de la hoja de vida	Se muestra “No se pudo generar el QR”
jsQR 1.4.0	cdnjs.cloudflare.com	Leer QR con la cámara cuando el navegador no tiene BarcodeDetector	El escáner puede no leer el QR
Bricolage Grotesque, Figtree	Google Fonts	Tipografía	Se usa Arial como respaldo
Ninguna biblioteca de interfaz	—	No se usan React, jQuery ni frameworks CSS	—

4.2 Funciones del navegador utilizadas

API	Para qué se usa
localStorage	Guardar la base de datos completa
FileReader y Canvas	Leer archivos y reducir las fotografías (máx. 800 px, JPEG con calidad 0,6)
Blob y URL.createObjectURL	Descargar JSON, CSV y SQL; abrir PDF guardados como datos
window.open y document.write	Generar reportes e informe técnico imprimibles en una pestaña nueva
getUserMedia / BarcodeDetector	Abrir la cámara y detectar el QR
Service Worker	Registro de <code>sw.js</code> al cargar la página

5. Modelo de datos

Todo el estado vive en un objeto `db` que se guarda como un único texto JSON en `localStorage`, bajo la clave `electrosafe_db`. Sus colecciones son:

Colección	Tipo	Campos
<code>eq</code>	Objeto (1 equipo)	marca, modelo, serie, fabricante, año, servicio, estado, tipo, aplicacion, modalidades, seguridad, gmdn, clasificacion, registro, vida, adquisicion, responsable (17)
<code>inv</code>	Lista	nombre, categoria, fabricante, referencia, serie, cant, estado, disp, prueba, ingreso, ultima, proxima, obs, foto (14)
<code>mant</code>	Lista	fecha, tipo, estado, resp, horas, repuestos, resultado, firma, desc (9)
<code>chk</code>	Lista	tipo, fecha, resp, obs, res = lista de pares [punto, respuesta]
<code>pru</code>	Lista	fecha, prueba, esperado, medido, unidad, instrumento, resultado, resp (8)
<code>ins</code>	Lista	nombre, verifica, instrumento, procedimiento, criterio, norma, fuente (7)
<code>fotos</code>	Lista	titulo, src (imagen en base64), fecha
<code>docs</code>	Lista	nombre, texto, pdf (PDF en base64)

Colección	Tipo	Campos
concl, interp	Texto	Conclusiones e interpretación del grupo
seeded, seedIns	Marcas	Evitan volver a cargar los datos iniciales

5.1 Datos iniciales (semillas)

Al primer ingreso se precargan: los datos de identificación del equipo (constante `HVD`), siete elementos de inventario (`INV0`) con estado “No verificado” y cuatro fichas de pruebas. Además, en cada carga, los campos vacíos de la hoja de vida se completan con `HVD`. Las constantes `CHK` (puntos de checklist), `RIESGOS`, `FAQ` y `KB` (base de conocimiento del asistente) forman parte del código y no se guardan en `localStorage`.

5.2 Relación con el modelo propuesto

El documento 05_Base_de_Datos propone un modelo relacional con claves primarias y foráneas. La plataforma implementa una versión plana y local de ese modelo:

Entidad propuesta	En la plataforma	Diferencia
Equipo biomédico	eq	Un solo equipo; sin <code>id_equipo</code>
Mantenimiento	mant	Agrega horas fuera de servicio, estado, repuestos y firma
Inspecciones	chk	Se implementan como checklists con puntos
Riesgos	Constante <code>RIESGOS</code>	Solo lectura; no hay formulario de registro
Documentos	docs y PDFS	Los PDF fijos se listan en el código; los cargados, en <code>localStorage</code>
Usuarios	—	No implementada

5.3 Exportación a SQL

El botón “Exportar script SQL” (pestaña Base de datos) genera un archivo con una sentencia `CREATE TABLE` y varias `INSERT` por tabla. Las tablas son: equipo, inventario, mantenimiento, pruebas_funcionales, instrumentos_pruebas, checklist, evidencias, documentos y checklist_items. Cada tabla tiene una columna `id INTEGER PRIMARY KEY` y el resto de columnas son de tipo `TEXT`. `checklist_items` relaciona cada punto con su checklist mediante `checklist_id`. El script no incluye fotografías ni PDF, y no define claves foráneas ni tipos numéricos o de fecha: si se llevan a un motor real, esas restricciones deben agregarse.

6. Persistencia y copias de seguridad

6.1 Lectura y escritura

- **Lectura:** al cargar, se hace `JSON.parse(localStorage.getItem(K))` y se combina con los valores por defecto con `Object.assign`.
- **Escritura:** `save()` llama a `localStorage.setItem`. Si el navegador rechaza la escritura (cuota llena), muestra una alerta pidiendo eliminar fotos o PDF pesados.

- **Límite:** localStorage ofrece del orden de 5 MB por origen (valor aproximado, depende del navegador). Las fotos (base64) y los PDF cargados son lo que más consume; por eso las fotos se reducen y los PDF se limitan a 1,8 MB.

6.2 Exportar, importar y cargar datos del repositorio

Acción	Mecanismo	Comportamiento
Exportar datos (JSON)	<code>JSON.stringify(db)</code> a un Blob descargable	Genera <code>electrosafe-datos.json</code> con todo el estado
Importar	Lee el archivo y aplica <code>Object.assign(db, json)</code>	Reemplaza cada colección presente en el archivo; no fusiona registros
Cargar datos del grupo	<code>fetch('datos/electrosafe-datos.json')</code> y el mismo <code>Object.assign</code>	Igual que importar; muestra un aviso si el archivo no existe

Importante: Al usar `Object.assign` sobre `db`, una colección del archivo importado sustituye a la local completa. Si se desea fusionar, habría que implementar una lógica de combinación por registro. Esta decisión es sencilla pero puede borrar datos locales si se importa un archivo antiguo.

7. Lógica de los módulos

7.1 Formularios definidos por datos

Los formularios de equipo, inventario, mantenimiento, pruebas e instrumentos no están escritos a mano: se generan a partir del objeto `D`, donde cada campo es una lista `[nombre, etiqueta, tipo, opciones]`. La función `armar()` convierte esa definición en campos de texto, números, fechas, áreas de texto, listas desplegables o carga de archivo. Para agregar un campo basta con añadirlo a `D`; aparecerá en el formulario, en la exportación SQL, en los reportes y en la tabla de la pestaña Base de datos.

7.2 Crear, editar y eliminar

- **Crear:** el manejador del formulario lee los valores con `FormData` y los agrega con `push`.
- **Editar:** el botón Editar copia el registro al formulario y marca `dataset.edi` con su posición; al guardar, esa posición se reemplaza.
- **Eliminar:** el botón guarda `colección:posición` en `data-del` y ejecuta `splice` sin pedir confirmación.
- Los registros se identifican por su **posición** en la lista, no por un identificador único.

7.3 Seguridad del contenido mostrado

La función `esc()` reemplaza los caracteres `&`, `<`, `>` y comillas dobles antes de insertar texto del usuario en las tablas y tarjetas, para evitar que un dato escrito como HTML se ejecute. La función `v()` aplica `esc()` y muestra “—” cuando el valor está vacío.

7.4 Checklist

Los puntos provienen de la constante `CHK` (10 preventivos y 6 correctivos). Al guardar se almacena la lista de pares `[punto, respuesta]`. Las respuestas válidas son OK/NO/N-A (preventivo) y Sí/No/N-A (correctivo). La conformidad es `aprobados / (aprobados + fallas)` y excluye los N/A. Para cambiar o ampliar la lista (por ejemplo, para igualar los 20 ítems del protocolo 14_Protocolo), se edita `CHK`; los checklists ya guardados conservan los puntos con los que se registraron.

7.5 Evidencias fotográficas

La función `foto()` lee la imagen, la dibuja en un *canvas* con el lado mayor limitado a 800 px y la exporta como JPEG con calidad 0,6 en formato *data URL*. Ese texto se guarda en el registro. Es eficiente en espacio, pero limita la cantidad de fotos que caben en `localStorage`.

7.6 Documentos y visor

El arreglo `PDFS` lista los 14 documentos del repositorio con su nombre y ruta. `pdfInline()` los muestra en un *iframe* de la misma página; si la ruta es un *data URL* (PDF cargado por el usuario), lo convierte antes a una URL de objeto. Para agregar un documento al visor, se añade una entrada a `PDFS` y se sube el archivo a la ruta indicada.

8. Indicadores de gestión

La función `ind()` calcula los indicadores del Dashboard y del informe técnico. Sea *R* el conjunto de mantenimientos con estado Realizado y *C* el subconjunto de tipo Correctivo:

Variable	Definición en el código
H (horas totales)	Máximo entre 24 y las horas transcurridas desde la fecha más antigua de <i>R</i> hasta el momento actual. Si <i>R</i> está vacío, $H = 0$
down (horas fuera de servicio)	Suma del campo <code>horas</code> de todos los mantenimientos en <i>R</i>
up (horas operativas)	Máximo entre 0 y $(H - \text{down})$
MTBF	$\text{up} \div \text{número de elementos en } C$ (si <i>C</i> está vacío, no se calcula)
MTTR	$\text{Suma de horas de los elementos de } C \div \text{número de elementos en } C$
Disponibilidad operativa	$\text{up} \div H \times 100$
Disponibilidad inherente	$\text{MTBF} \div (\text{MTBF} + \text{MTTR}) \times 100$
Cumplimiento PM	$\text{Preventivos realizados} \div \text{todos los preventivos (realizados + programados)} \times 100$

Consideraciones de diseño que conviene conocer al sustentar:

- La ventana de tiempo (H) depende de la fecha actual, por lo que los indicadores varían con los días aunque no cambien los registros.
- El MTBF solo cuenta como fallas los mantenimientos correctivos en estado Realizado; los preventivos no cuentan como falla pero sí suman sus horas fuera de servicio a `down`.
- Los tiempos fuera de servicio de los preventivos también reducen las horas operativas (up).
- Si no hay datos suficientes, el indicador muestra “—” en lugar de 0, para no inducir a error.

El cronograma del proyecto distingue MTBF de MTTF; en la plataforma se implementa MTBF (equipos reparables), coherente con la sección de mantenimiento correctivo.

9. Asistente IA (búsqueda sobre la documentación)

El “Asistente IA” **no usa un modelo de lenguaje ni llama a un servicio externo**: es un buscador de fragmentos que funciona dentro del navegador. Su diseño garantiza que solo responda con información incorporada y siempre cite la fuente. Funciona así:

- 1 **Normalización.** La pregunta se pasa a minúsculas, se eliminan tildes y se separa en palabras de más de 2 letras que no sean palabras vacías (STOP).
- 2 **Términos.** Se aplican sinónimos (por ejemplo, “sirve” → “utiliza”) y se toma la raíz de las palabras (primeras 5 letras). Las palabras genéricas como “equipo”, “valleylab” o “electrobisturi”, incluso con errores de escritura (distancia de Levenshtein ≤ 2), pesan menos.
- 3 **Fragmentos.** Las fuentes se dividen en párrafos y estos en trozos de hasta 700 caracteres. Las fuentes son: la base $\mathbb{KB}$ (12 entradas), los documentos cargados por el usuario y las fichas de la pestaña Instrumentos y pruebas que tengan contenido.
- 4 **Puntuación.** Cada término recibe un peso tipo IDF, $\ln(1 + N / (1 + df))$, que premia los términos poco frecuentes. Un fragmento suma el peso de los términos que contiene, con un bono por repeticiones y otro si el término aparece en el título del documento.
- 5 **Filtro y respuesta.** Se exige que el fragmento contenga al menos la mitad del peso de los términos específicos. Se devuelven hasta dos fragmentos cuya puntuación sea al menos el 60 % de la mejor, con su fuente. Si no hay ninguno, responde que no encontró la información.

Pendiente del grupo: Tres entradas de $\mathbb{KB}$ están rotuladas como “borrador”: consumo de corriente, instrumentos necesarios y corriente de fuga/resistencia de tierra. No provienen de los documentos del grupo y deben verificarse con el manual de servicio antes de la sustentación. En la declaración de uso de IA debe constar que el asistente es un buscador local y qué contenido fue redactado con apoyo de herramientas de IA.

10. Código QR de la hoja de vida

10.1 Generación

La función `qr()` toma nueve campos de la hoja de vida (marca, modelo, serie, fabricante, año, servicio, estado, registro y GMDN), elimina cualquier texto entre paréntesis, los convierte a JSON y los codifica en Base64. El resultado se agrega a la dirección actual de la página como fragmento: `plataforma.html#hv=<base64>`. Ese enlace se dibuja como QR de 240 × 240 con corrección de errores nivel L (la más baja, que permite más datos).

10.2 Lectura

Al cargar la página o cambiar el fragmento, `verHV()` decodifica el contenido y lo muestra en la franja amarilla superior. El botón flotante abre la cámara con `getUserMedia` (cámara trasera) y analiza los cuadros: usa `BarcodeDetector` si el navegador lo ofrece, y en caso contrario `jsQR` sobre un *canvas*. Solo se aceptan textos que contengan `#hv=`.

Nota: Como los datos viajan dentro del enlace, el QR funciona en cualquier celular sin conexión a la base local, pero solo si la página está publicada con una dirección accesible. La cámara exige HTTPS (GitHub Pages lo cumple). Si se cambia la hoja de vida, hay que regenerar e imprimir de nuevo el QR.

11. Reportes e informe técnico

- **Reportes** (pestaña Reportes): cuatro opciones (mantenimiento, pruebas, inventario, instrumentos). El botón Ver/imprimir abre una pestaña con una tabla HTML lista para imprimir o guardar como PDF.
- **CSV:** separador punto y coma, comillas dobles escapadas y marca BOM (UTF-8) para que Excel muestre bien las tildes. El archivo se llama `electrosafe-<reporte>.csv`.
- **Informe técnico** (Dashboard): hoja imprimible con nueve secciones: identificación, indicadores, historial, checklists, pruebas, inventario, matriz de riesgos, evidencias y conclusiones. Las conclusiones combinan un resumen automático de cifras con los textos que el grupo escribió.

Algunos navegadores bloquean las ventanas emergentes; si el reporte no se abre, hay que permitir las para el sitio.

12. Aplicación instalable (PWA)

Ambas páginas enlazan el archivo `manifest.json`, declaran el color de tema `#06293d`, usan `icon-192.png` como icono y registran el `service worker` con `navigator.serviceWorker.register("sw.js")` al evento *load*. Con esto el navegador ofrece instalar la aplicación en el dispositivo.

Pendiente del grupo: No se revisó el contenido de `manifest.json` ni de `sw.js` al preparar este manual. El grupo debe completar aquí: (1) los campos del manifest (`nombre`, `start_url`, `display`, `iconos`) y (2) la estrategia de caché del service worker y la lista de archivos que guarda. Si el worker guarda en caché las páginas, una versión nueva puede tardar en verse: por eso se recomienda cerrar y abrir la app dos veces, o cambiar el nombre/versión de la caché en cada publicación.

13. Seguridad y privacidad

Aspecto	Estado actual	Recomendación
Autenticación y roles	No existe	Requiere backend; es el principal trabajo futuro
Confidencialidad de los datos	Los datos quedan sin cifrar en el navegador del dispositivo	No registrar datos personales de pacientes; usar dispositivos propios
Inyección de HTML	Los textos de los usuarios se escapan con <code>esc()</code> al mostrarse	Mantener <code>esc()</code> en cualquier campo nuevo
Datos en el QR	Nueve campos de la hoja de vida, legibles por quien escanee	No incluir información sensible en la hoja de vida
Integridad de registros	Se puede editar o eliminar cualquier registro; sin auditoría	Exportar copias periódicas; a futuro, registro de cambios
Scripts de terceros	Se cargan de cdnjs sin verificación de integridad	Agregar atributos <i>integrity</i> o alojar las bibliotecas en el repositorio

14. Despliegue en GitHub Pages

14.1 Publicación

- 1 En el repositorio (github.com/sun-max13/Electro_Safe, verificar el usuario), use **Add file** → **Upload files** y suba a la raíz: `index.html`, `plataforma.html`, `manifest.json`, `sw.js`, `icon-192.png` e `icon-512.png`.
- 2 Suba los PDF a `documentos/` y `img/` respetando exactamente los nombres. Suba los archivos sueltos, no un `.zip`.
- 3 En **Settings** → **Pages**, elija como fuente la rama principal y la carpeta raíz. Guarde.
- 4 Espere uno o dos minutos y abra la dirección del sitio. Recargue con `Ctrl + F5`.

14.2 Verificación posterior a la publicación

Verificación	Resultado esperado
Abrir <code>index.html</code> y pulsar “Plataforma”	Se abre la plataforma con las 13 pestañas
Pestaña Manuales: abrir cada documento	Todos los PDF se ven en el visor (sin pantalla en blanco)
Guardar una hoja de vida y revisar el QR	El QR aparece; escaneado, muestra los datos
Registrar un mantenimiento correctivo realizado con horas	Dashboard muestra MTBF y MTTR
Exportar datos (JSON) e importarlo	Los registros se restauran
Asistente: “¿Qué es el sistema REM?”	Devuelve un fragmento con su fuente
Abrir desde el celular e instalar	La app se instala y abre a pantalla completa

Esta lista es una guía de verificación manual; este manual no certifica que las pruebas se hayan ejecutado. El grupo debe ejecutarlas y dejar constancia (por ejemplo, con las evidencias fotográficas de la plataforma).

15. Guía de modificación del código

Quiero...	Qué modificar
Agregar un campo a un módulo	Añadir la entrada correspondiente al objeto <code>D</code> (por ejemplo, <code>D.mant</code>). Aparece en formulario, reportes y SQL
Cambiar los puntos del checklist	Editar <code>CHK.Preventivo</code> o <code>CHK.Correctivo</code> . Cada punto es <code>[texto, norma]</code>
Completar la matriz de riesgos	Editar <code>RIESGOS</code> : cada fila es <code>[riesgo, P, S, nivel, clasificación, control]</code> . Las clases de color son <code>r-Bajo</code> , <code>r-Moderado</code> , <code>r-Alto</code> y <code>r-Crítico</code>
Agregar un PDF al visor	Añadir <code>[nombre, ruta]</code> a <code>PDFS</code> y subir el archivo
Ampliar lo que sabe el asistente	Añadir objetos <code>{nombre, texto}</code> a <code>KB</code> , o cargar un <code>.txt</code> desde la pestaña Manuales
Cambiar los datos iniciales del equipo	Editar <code>HVD</code> (solo afecta campos vacíos)
Reiniciar todos los datos de prueba	Borrar los datos del sitio en el navegador, o ejecutar <code>localStorage.removeItem("electrosafe_db")</code> en la consola
Agregar una pestaña	Añadir un botón en <code>#tabs</code> con <code>data-tab</code> y una <code>section.tab</code> con el mismo id

Después de cualquier cambio, vuelva a publicar el archivo y fuerce la recarga (Ctrl + F5). Si la PWA está instalada, siga las indicaciones de la sección 12.

16. Limitaciones y trabajo futuro

Limitación actual	Mejora propuesta
Datos locales a cada navegador, sin sincronización	Backend con base de datos (por ejemplo, SQL) y API; usar el script SQL como punto de partida
Sin autenticación ni roles	Inicio de sesión y control de acceso por rol (ingeniero, técnico, usuario, administrador) según los casos de uso
Un solo equipo	Agregar identificador de equipo y relaciones de clave foránea
Registros identificados por posición; eliminar sin confirmar	Identificadores únicos, confirmación y registro de auditoría
Importar reemplaza colecciones completas	Fusión por identificador
Matriz de riesgos solo de lectura y sin valores P/S	Formulario de riesgos y cálculo automático del nivel ($P \times S$)
Checklist y mantenimiento sin vínculo	Asociar el checklist a la intervención que lo origina
Asistente basado en palabras clave	Integrar un modelo de lenguaje con recuperación de documentos (RAG), manteniendo las citas de fuente
Almacenamiento limitado (~5 MB)	IndexedDB para fotos y PDF, o almacenamiento en servidor
Programación y alertas de mantenimiento no implementadas	Calendario con avisos según la "próxima inspección"

17. Documentos de referencia del proyecto

Documento	Relación con este manual
03_Aplicacion, 04_Modulos	Alcance funcional y módulos propuestos
05_Base_de_Datos	Modelo relacional propuesto (sección 5.2)
06_Casos_de_uso y diagrama-uso	Actores y casos de uso que guían la funcionalidad
07_Arquitectura	Capas y proyección de backend (sección 2)
09_Mantenimiento y 14_Protocolo	Checklists, frecuencia anual propuesta y matriz de riesgos P × S
10_Riesgos	Matriz y medidas de control
12_Manual_Servicio_Force_FX	Referencia del fabricante para pruebas y criterios de aceptación
diagrama-bloques, diagrama-flujo	Funcionamiento del equipo

Este documento es una descripción académica del software; no sustituye el manual de servicio del fabricante ni los procedimientos institucionales.